

IoT and Smart Home, RFID

☀ What is RFID?

RFID is a technology that lets you **read data from a card or tag without touching it**—you just bring it near a **reader**. **Radio Frequency Identification**.

🧩 What are the parts?

1. **RFID Reader** (like the RC522 module):
 - Sends out a **magnetic field**.
 - Reads the information from the tag when it comes close.
 2. **RFID Tag or Card**:
 - Has **no battery** (it's called a *passive* device).
 - Inside it are:
 - A **tiny chip** (stores data like an ID number),
 - An **antenna** (to send and receive signals).
-

⚡ How does it work?

1. The **reader** creates a **magnetic field** around it.

2. RFID Tag (or card)

- Has:
 - A small **chip** with data (like an ID number).
 - A **coil antenna** (to receive energy from the reader).
 - **No battery!** It gets energy from the reader's magnetic field.
-

⚙️ How It Works (with Diagram & Physics)

Step 1: Magnetic Field Generates Current in Tag

The reader creates a **changing magnetic field** using **alternating current (AC)** in a coil.

Formula 1: Faraday's Law of Induction

$$\mathcal{E} = -\frac{d\Phi_B}{dt}$$

- $\mathcal{E}$ = induced voltage in the tag's coil
- Φ_B = magnetic flux through the coil
- $\frac{d\Phi_B}{dt}$ = change in magnetic flux over time

This tells us that a **changing magnetic field** can **generate voltage** (electricity) in the RFID tag's coil.

$$\mathcal{E} = -\frac{d\Phi_B}{dt}$$

Where:

- $\mathcal{E}$ = induced electromotive force (voltage, in volts)
- Φ_B = magnetic flux = $B \cdot A \cdot \cos \theta$
 - B = magnetic field strength (in teslas, T)
 - A = area of the loop (in m²)
 - θ = angle between the field and the loop's normal
- $\frac{d\Phi_B}{dt}$ = how fast the magnetic flux changes over time
- The **negative sign** is **Lenz's Law**: the induced current **opposes the change** in magnetic flux.

2. When you bring the **tag close**, the magnetic field causes a small electric current inside the tag
3. That small amount of electricity **powers the tag's chip**.
4. The tag then **sends its data back** to the reader using radio waves.

Once the tag is powered, it **modulates** (changes) the signal to **send data back** to the reader.

This uses a method like **load modulation**:

- The tag quickly switches a tiny resistor on and off to cause changes in the magnetic field that the reader can detect.

Electromagnetic wave frequency:

$$f = \frac{c}{\lambda}$$

- f = frequency of the radio wave
- c = speed of light (3×10^8 m/s)
- λ = wavelength of the signal

RFID readers often operate at **13.56 MHz** (for RC522), which is a **high-frequency radio wave**.

No single person wrote $f = \frac{c}{\lambda}$ first, but it comes naturally from the **wave speed formula**:

$$\text{wave speed} = \text{frequency} \times \text{wavelength}$$

This was known from **general wave theory**, even **before** Maxwell. But once Maxwell showed that **light is an electromagnetic wave**, this formula applied to light and radio waves too.

⚡ Who contributed?

1. James Clerk Maxwell (1831–1879)

- He developed the **Maxwell equations**, which describe how electric and magnetic fields work and interact.
- He was the first to **predict that light is an electromagnetic wave** traveling at speed c .
- From Maxwell's equations, the wave speed is:

$$c = \frac{1}{\sqrt{\mu_0 \epsilon_0}}$$

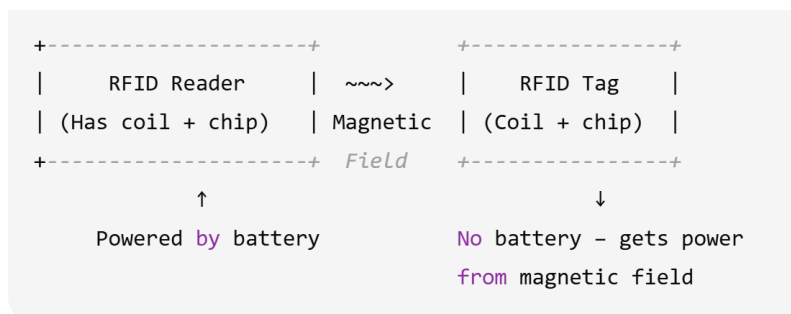
where:

- μ_0 = permeability of free space
- ϵ_0 = permittivity of free space

2. Wave physicists (1600s–1800s) like Christiaan Huygens, Thomas Young, and Augustin-Jean Fresnel

- Helped develop the concept of **wave properties** like **wavelength** and **frequency**.

5. The **reader gets the data** and can use it—like identifying who you are or unlocking a door.



Digital Communication Protocols — Not Physics Formulas

After the RFID tag sends its data (like your ID number), the **reader decodes it** using a specific **communication protocol**, like:

- **ISO/IEC 14443** (for contactless smart cards like school badges, metro cards)
 - **SPI or I²C** (if you're using an RC522 RFID module with a microcontroller)
-

Imagine the tag sends:

Tag ID = [0xDE, 0xAD, 0xBE, 0xEF]

The reader receives this binary or hexadecimal data and:

1. **Reads the tag ID**
2. **Compares it to a stored list** (e.g., in a database or lookup table)
3. **Identifies the person or object**

0xDE is hexadecimal (base-16) notation.

What does that mean?

- **Hexadecimal** is a number system with **base 16**.
- It uses **digits 0–9** and **letters A–F**:
 - A = 10, B = 11, ..., F = 15

So:

0xDE = (13 × 16) + 14 = 208 + 14 = 222 in decimal

- The prefix **0x** just tells you: "Hey, this number is in hexadecimal."

```
allowed_ids = {
    "DEADBEEF": "Jessica Smith",
    "A1B2C3D4": "John Doe"
}

if tag_id in allowed_ids:
    print("Welcome,", allowed_ids[tag_id])
else:
    print("Access Denied")
```

The **RFID reader (MFRC522)** is controlled by writing/reading these **registers**.

It uses **commands** to send radio signals to the tag and get data back.

The tag's **UID** is captured and stored using the **Uid** class.

Commands like **REQA** initiate communication, while others handle selecting and reading the tag.

Status codes help software know if things went well or if there were communication issues

How This Works In Practice:

- 1. Initialization:**
Your program writes to these registers (using these constants) to set up the reader.
- 2. Tag Detection:**
The reader sends a **PICC_CMD_REQA** command to ask if any tag is nearby.
- 3. Collision Avoidance:**
If multiple tags respond, the reader uses anti-collision commands (like **PICC_CMD_SEL_CL1**) to select one tag and get its UID.
- 4. UID Retrieval:**
The reader receives the tag's UID bytes and stores them in the **Uid** object (**uidByte** list and **size**).
- 5. Authentication and Data Exchange:**
Using commands like **PICC_CMD_MF_AUTH_KEY_A** and **PICC_CMD_MF_READ**, the reader authenticates and reads/writes data blocks on the tag.

6. Status Handling:

The software checks the status codes after each operation to handle errors, retries, or success.

`mfr522_i2c.py`

This code is a Python class (`mfr522`) that combines:

- `mfr522Config` — a class defining the RFID chip's registers and commands (like constants and instructions),
- `softIIC` — a class that implements **software I²C (Inter-Integrated Circuit)** communication, a protocol used to talk to the RFID chip over two wires (SCL and SDA).

SDA and **SCL** are the two fundamental wires used in the **I²C (Inter-Integrated Circuit)** communication protocol.

What they do:

Wire	Full Name	Purpose
SDA	Serial Data Line	Carries the data bits between devices
SCL	Serial Clock Line	Carries the clock signal generated by the master to synchronize data transfer

How they work together:

- **SCL** is controlled by the master device and sets the timing for data transfer.
- On each clock pulse on **SCL**, data bits are sent or read over the **SDA** line.

- Both wires are **open-drain/open-collector**, which means devices can only pull the lines low; external **pull-up resistors** keep the lines high when no device is pulling them down.

Absolutely — let's take a deeper look at **I²C (Inter-Integrated Circuit)** and **why SDA and SCL are essential** for it to work.

What Is I²C?

I²C is a **serial communication protocol** that allows multiple "slave" devices (like sensors, displays, or RFID readers) to communicate with one "master" device (like a microcontroller) over **just two wires**.

It's widely used in embedded systems because it's:

- Simple (only 2 wires needed)
 - Supports multiple devices on the same bus
 - Bidirectional (can read and write data)
-

Key Concepts of I²C

1. SCL (Serial Clock Line) – Generated by the master

- This line provides the **timing** signal.
- Every bit of data on the SDA line is synchronized with a clock pulse from SCL.
- Without SCL, devices wouldn't know **when** to read or write.

2. SDA (Serial Data Line) – Used for sending/receiving bits

- Carries the **actual data** (address, commands, responses, etc.).
- Devices put data on SDA and read from it **in sync with SCL**.

3. Pull-up resistors (typically 4.7kΩ) are required on both lines

- Both lines are **open-drain**, meaning devices can only pull them **low**, not high.
 - Resistors "pull" the line **high** when no device is pulling it low.
-

How Communication Happens

Step-by-step:

1. **Start condition:** Master pulls SDA low **while** SCL is high — tells all devices "a message is coming".
2. **Address phase:** Master sends a 7-bit address (e.g. of the RFID reader) + 1-bit for Read/Write.
3. **ACK/NACK:** The addressed slave (RFID) responds by pulling SDA low (ACK = Acknowledge).
4. **Data transfer:** Bits are sent or received, 1 per SCL clock cycle, over SDA.
5. **Stop condition:** Master releases SDA to high **while** SCL is high — communication ends.

Every step **requires both SDA and SCL working together**.

! Why Both Lines Are Needed

Reason	Explanation
Timing	Data on SDA is only valid when timed with clock pulses on SCL.

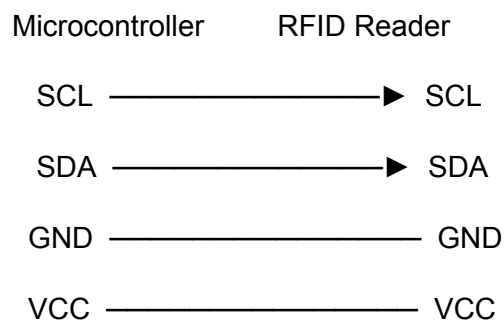
Synchronization	SCL ensures both master and slave stay in sync — they "agree" when bits start and stop.
Error-free data	Without a clock signal (SCL), bits on SDA could be misread or missed entirely.
Protocol compliance	The I ² C protocol depends entirely on this timing — you can't send data without it.

In Your Case (RFID)

If you're using an RFID module that supports I²C (like a version of **MFRC522 with I²C interface**):

- **SCL must be connected** to the microcontroller's SCL pin (often labeled D22 or GPIO22 on ESP32).
 - **SDA must be connected** to the corresponding SDA pin (often D21 or GPIO21).
 - You'll need to use an I²C library to talk to the RFID chip — it handles all the clocking and timing automatically.
-

Diagram



Both SDA and SCL must be connected **correctly and consistently**, with matching voltages and pull-up resistors in place.

Summary

- **SCL** provides the timing that makes I²C possible.
- **SDA** carries the actual data.
- Both are absolutely required for I²C to function — **remove one and the protocol breaks**.
- In an I²C RFID setup, like using MFRC522 with I²C firmware, you must connect **both lines** for any communication to occur.

Why SLC is important?

The slave sees a constant or floating data line (**SDA**) — but has no idea **when** a bit starts or ends.

The slave **can't decode anything**, even if data is technically "being sent."

It's like trying to listen to Morse code without the timing — you'd just hear beeps, but not know when one letter ends and the next starts.